GENAI MATURITY

From demo to reliable production and agentic engineering

Gilbert Pochet

Future Secure AI (FSAI)

Staff AI Engineer

THE GOAL OF THIS TALK

NOT LESSONS. SOMETHING YOU CAN APPLY.

After tonight, I want you to think twice before you...

1. **...accept an AI project.**
   Which decision changes? Which OKR does it move?

2. **...commit to a number.**
   How will you know you actually hit it?

3. **...say "it's easy, I can do it with AI."**
   A demo is not production.

Jeffrey Pfeffer, *Leadership BS* (2015), ch. 1: "Why Inspiration and Fables Cause Problems and Fix Nothing"

1. Who has built an AI product in a weekend?
2. Who has one in production, with a business metric attached?

FOUR STEPS OF GENAI MATURITY

01 · DEMO

Prototype

”This looks useful.”

02 · VALUE

Prove impact

”This creates measurable value.”

03 · RELIABILITY

Production

”It keeps creating value, every day.”

04 · SPEED

Agentic engineering

”We deliver it faster, without losing quality.”

IMPRESSIVE DEMOS ARE EASY. PRODUCTION VALUE IS NOT.

A DEMO NEEDS

- An API key
- A prompt
- A nice UI
- Five hand-picked examples

PRODUCTION NEEDS

- Edge cases
- Latency & cost
- Security & privacy
- Evaluation
- Monitoring
- User trust

AND THE PEOPLE AND PROCESS TO GET THERE

- A real development lifecycle
- The right engineers
- The right leader
- A team set up to succeed

AND PROOF IT IS WORTH IT

- A baseline
- The right impact metric

THE 80% ILLUSION



1 · THE PROTOTYPE TRAP

THE RESULT: PILOT PURGATORY

Endless pilots. Nothing in production.

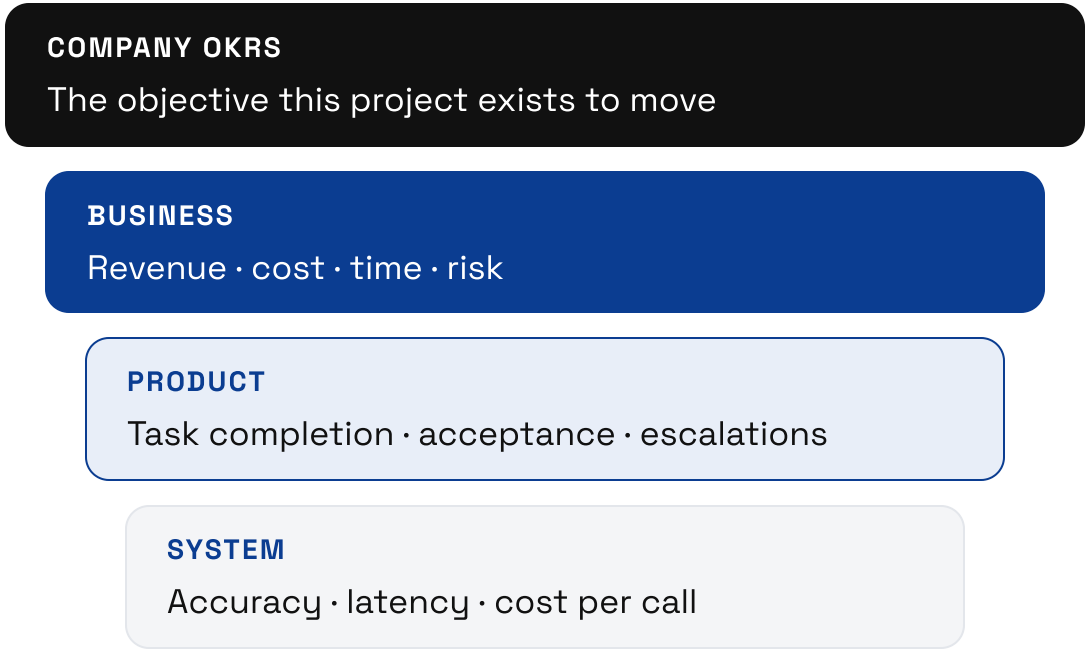
- No clear owner
- No baseline
- No evals
- Surprise costs at scale
- Risk review too late

START FROM THE DECISION, NOT THE MODEL

~95% of enterprise GenAI pilots show no measurable P&L impact.
MIT NANDA, *The GenAI Divide*, 2025

#1 root cause of AI project failure: the wrong problem, so models optimise the wrong metric.
RAND, *Root Causes of Failure for AI Projects*, 2024

1. **What decision or process changes if this works?**
   If nothing changes, there's no value to measure.
2. **What does it cost today?**
   The baseline: time, errors, conversion, cost per task.
3. **What has to move, and by how much?**
   How much speed, revenue or cost are we actually winning?
4. **Which OKR does it serve?**
   If it isn't tied to a company objective, it won't survive prioritisation.



Each layer must explain the one above it.

SIX MONTHS, FOUR PEOPLE, NO RELEASE

Before GenAI: data science, transformers, deep learning.

The goal: match candidates' CVs to the right jobs.

Projects ran 6+ months with a team of four.

Solid work. Never released.

WHAT WE MEASURED

How good the match was

Model metrics on the match itself.

WHAT WE NEVER DEFINED

What a good match was worth

To the recruiter, the customer and the business.

Not bad work. The wrong finish line.

THE QUESTIONS WE SHOULD HAVE ASKED FIRST

TIME

How much time does a recruiter save per match?

Value in a unit the business already tracks.

DEMAND

Which industries want this most?

Where adoption and impact come first.

FIT

Which industries don't benefit at all?

Some have so many candidates they don’t need matching.

AND THE PROMISES

”We’ll increase applications by 10%.” Or was it 5%? **Do we really get 10%? How do we know? Can we commit to it this quarter, and does that metric represent what the team will actually deliver?**

> Model metrics tell you it works. Business metrics tell you it matters.

TEST THE THEORY BEFORE BUILDING THE MODEL

1. **The premise**

   Recommend items before checkout: a better experience, and bigger orders.

2. **The test**

   Compare restaurants that already had great *manual* recommendations with restaurants that had none.

3. **The insight**

   The human version is the prototype. If manual recommendations move the number, AI can replicate them at scale.

40%

of menu items had no recommendations at all

- Restaurants didn't want to set them up
- Online menus weren't kept fresh
- Doing it by hand takes too long

THE DATA SHOWED WHERE THE VALUE WAS

Cart spend compared with restaurants that had no recommendations

FULL RECOMMENDATIONS

+20–30%

higher cart spend

ABOUT HALF COVERED

<5%

barely moves

NO RECOMMENDATIONS

Baseline

a lost opportunity on every order

Usability

An easier way to find what to add.

Automation

Restaurants get recommendations without the manual work.

Lost opportunity

40% of items with nothing to recommend: the biggest impact.

ONE PRIMARY METRIC, TIED TO THE OKR

PRIMARY METRIC

Cross-sell spend per order

SECONDARY

Time saved for restaurants · automation

Why only one? Marry too many metrics and nothing is objective. One metric explains how you support the OKR.

3 MONTHS

Build

BETA · 15 RESTAURANTS

+10% cross-sell

FULL ROLLOUT · 2 MONTHS LATER

+23–24% cross-sell

COMPANY OKR

More spend per shopping cart

3 · BUILD FOR PRODUCTION · MY GOLDEN RULE

An AI system is only as good as the software engineering around it.

We overestimate the model and underestimate the software.

AGENTIC ENGINEERING

THE AI LIFECYCLE: THEN AND NOW

Then

Data science era

Business metrics → Data & orchestration → Experimentation → Model & registry → Staging → Production & monitoring

4+ months to production

Now

GenAI + agentic engineering

Business metrics → Demo in days → Staging, early → Measure vs. the metric → Production → Monitoring

~1 month to production

> What changed is speed: we find out much earlier whether we're on track for the business goal.

THE ENGINEER'S ROLE HAS ALREADY CHANGED

YESTERDAY

~~WRITING THE CODE~~

- Features & boilerplate
- Tests & refactors
- Documentation

Agents do this now

→

TODAY

OWNING THE DECISIONS

- Architecture
- Flows & boundaries
- Keeping it simple, not over-engineered
- Accountability for what ships

→

NEXT

STILL SHIFTING

- Less typing, more judgement
- More direction, less execution

THE ONE CONSTANT

Models change every year. In 12 years of AI, one thing never has: the human in the loop.

AGENTIC ENGINEERING FRAMEWORK

ENGINEER	1 **Set the direction** A task, a Jira ticket or a bug fix		3 **Approve the plan** No code before "go"		5 **Review & merge** Peer review + automated checks
AGENT		2 **Read & plan** Code, rules & ADRs → phased plan + diagram		4 **Build in stages** Proof (tests) at every stage	
WHY	Left alone, agents reverse decisions and invent details. Anthropic, Project Vend, 2025	Doing too much at once leaves a mess: one task, progress written down. Anthropic, long-running agents, 2025	**~39% worse** in long multi-turn work. An early wrong turn rarely recovers. Laban et al., 2025	Long runs drift, and agents say "done" before it is. Vending-Bench, 2025 Anthropic, 2025	Anything with real consequences needs human approval. Anthropic, agentic misalignment, 2025

Follows Anthropic's own guidance: Best practices for Claude Code (explore, plan, code, commit) · Effective harnesses for long-running agents

PLAN BEFORE CODE: THE DEVELOPMENT PLAN

DEVELOPMENT_PLAN.md

```
# Development Plan for invoice-extraction

## Project Purpose and Goals
## Hard Requirements
## Unknowns and Assumptions
## Architecture Considerations

## Development Phases
### Phase 1: Context Gathering and Project Review
- [ ] Read all relevant project files
- [ ] Review existing tests and expected behaviour
### Phase 2-3: Design validation, preparation …
### Phase 4: Implementation Tasks
- [ ] Ensure implementation follows project conventions
### Phase 5: Testing and Validation
## QA Checklist
## Out of Scope
## Stop Point
Stop after creating this plan and wait for user review.
```

Context first

The agent reads code, docs and tests before planning.

Extend, don’t rewrite

Reuse existing patterns. Architecture is preserved by default.

Unknowns made explicit

No silent guessing to fill the gaps.

Checkable phases & QA gates

Progress you can verify, not ”trust me”.

Stop point

A human approves before any code is written.

Use the skill

`github.com/gilbertofp16/gen_ai_repo`

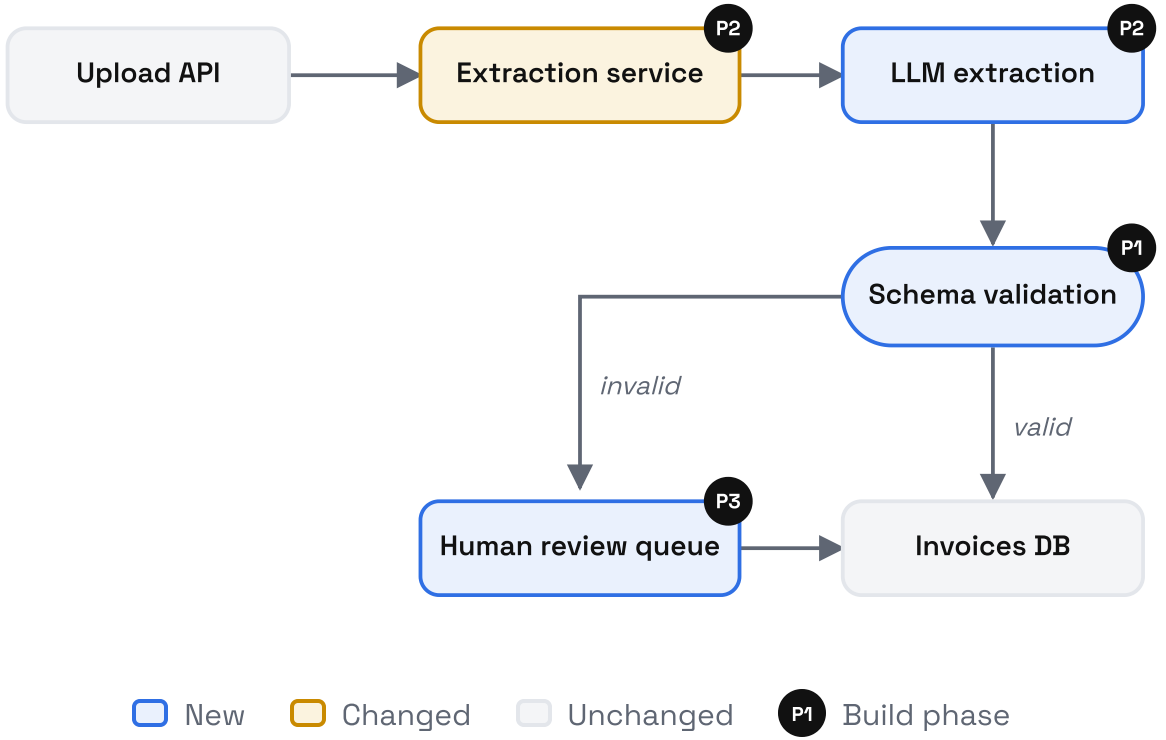
→ `Claude/skills/development-planner`

SEE THE PLAN BEFORE YOU APPROVE IT

plan-diagram.mmd

```
flowchart LR
  A[Upload API] --> B[Extraction service]:::changed
  B --> C[LLM extraction]:::new
  C --> D{Schema validation}:::new
  D -- valid --> E[(Invoices DB)]
  D -- invalid --> F[Human review queue]:::new
  F --> E

%% Phase 1: D · Phase 2: B, C · Phase 3: F
```



> Approve the picture before the code. Reviewers know what to expect, and the PR gets reviewed faster.

RUN THE PLAN, STAGE BY STAGE

1

BUILD ONE STAGE

Code + unit tests for the next phase only.

2

PROVE IT

Unit tests pass, manual testing when needed. Evidence, not ”done”.

3

APPROVE & SHIP

The PR carries the plan and the diagram, so review is fast. Normal CI/CD runs.

↺ Loop 1 → 2 until the task, ticket or bug fix is done

Engineering hasn't changed. CI/CD is still there. We just execute faster, so people review and decide instead of writing every line.

WHAT TO TAKE HOME

1	BEFORE YOU ACCEPT A PROJECT **Define the metric of success.**	One primary metric, tied to an OKR.
2	BEFORE YOU COMMIT TO A NUMBER **Investigate first.**	Test the theory with what exists today. That's how we found the 40%.
3	BEFORE YOU SKIP A STEP **The lifecycle hasn't changed. It got faster.**	Same steps: metrics, staging, production, monitoring. ~1 month instead of 4+.
4	BEFORE YOU SAY "IT'S EASY WITH AI" **Own the decisions, not the code.**	Architecture, flows and keeping it simple. That's the engineer's job now.
5	WHEN YOU BUILD WITH AGENTS **Plan first, then build in proven stages.**	Plan and diagram, human approval, tests at every stage. CI/CD unchanged.

THANK YOU

Questions?

Gilbert Pochet · github.com/gilbertofp16